

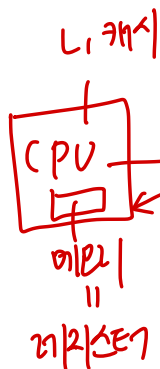
제3장

기억장치 관리

Ch 03. 기억장치 관리

목차

1. 개요



2. 기억장치의 계층구조 및 관리 기법

3. ① 단일 사용자 연속 기억장치 할당

↳ 힙/스택

강단점 //

4. ② 고정 분할 기억장치 할당

↳ 임그레이드

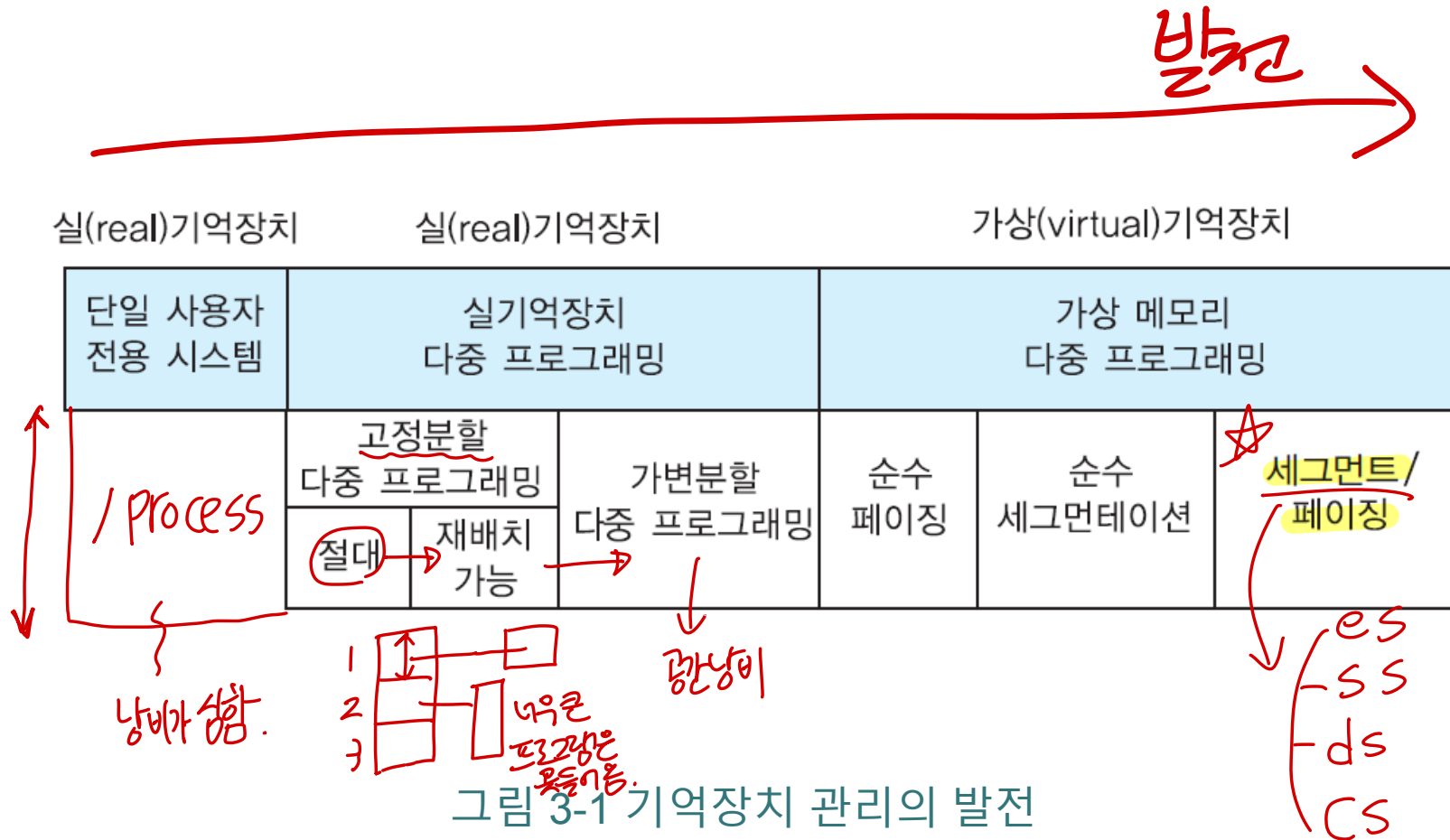
5. ③ 가변 분할 기억장치 할당

6. 기억장치 교체(swapping)

캐쉬: 좀더 빠른 메모리
디스크 버퍼링하는 곳

3. | 개요

발전



개요

- 주소 바인딩(address binding)

- 정의

- 논리적 주소(logical address)가 실행되기 위해 물리적 주소로 사상(mapping) 됨을 말함. 바인딩 시점에 따라 구분됨

1. 컴파일 시간(compile time) 바인딩

- 컴파일 시 절대 위치가 결정되는 경우의 바인딩 → 불변

2. 적재 시간(load time) 바인딩

- 프로그램이 기억장치에 적재되는 시간에 로더(loader)에 의해 이루어지는 경우의 바인딩
→ 읽을 쓰는 방식

3. 실행 시간(execution time) 바인딩

- 실행되는 동안에 기억장치의 한 세그먼트에서 다른 세그먼트로 옮겨질 경우와 같이 실행시간에 이루어지게 되는 경우의 바인딩

→ 부득이한 경우가 아니라면 하지 않음

개요

- 논리적 주소와 물리적 주소

가상

- 논리적 주소

- 가상주소(virtual address) : 공간상의 주소
 - 중앙처리장치가 생성함

참조

- 물리적 주소

- 메모리의 실제 주소, 즉 기억장치가 취급하는 주소

- 기억장치 관리기(MMU : Memory Management Unit)

- 논리적 주소를 물리적 주소로 변환

개요

4장에서 자세히

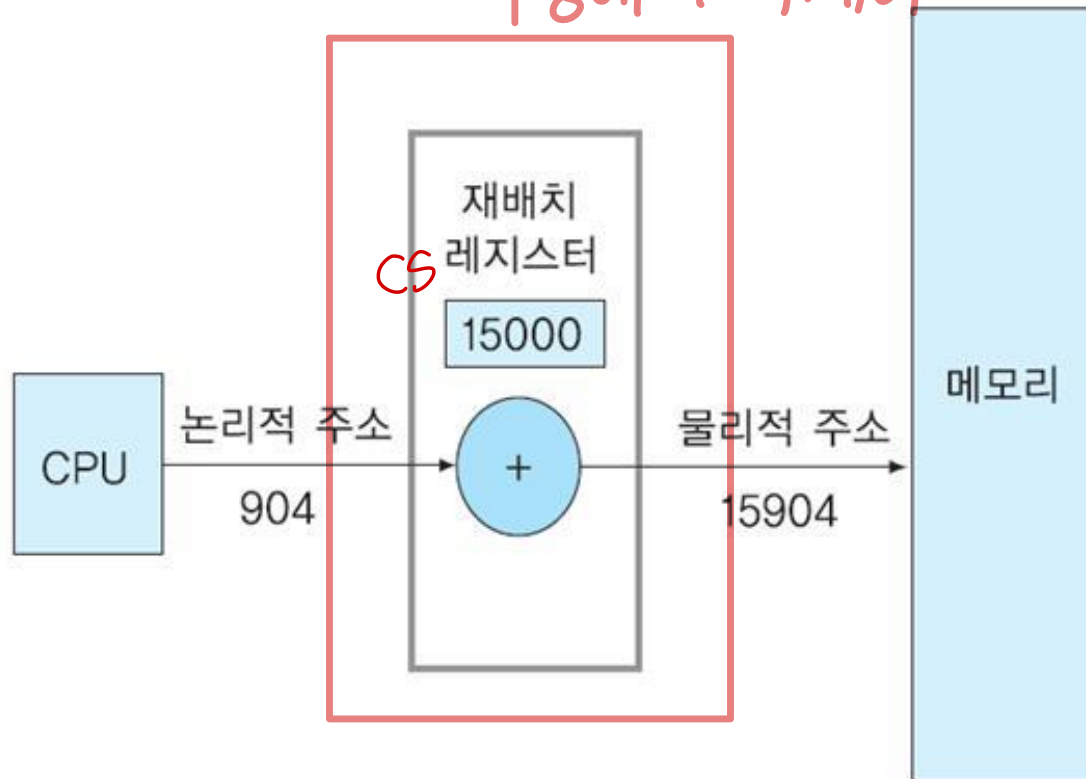


그림 3-2 재배치 레지스터를 이용한 주소변환

3.2

기억장치의 계층 구조 및 관리 기법

- 프로그램과 데이터가 실행되거나 참조되기 위해서는 주 기억장치에 있어야

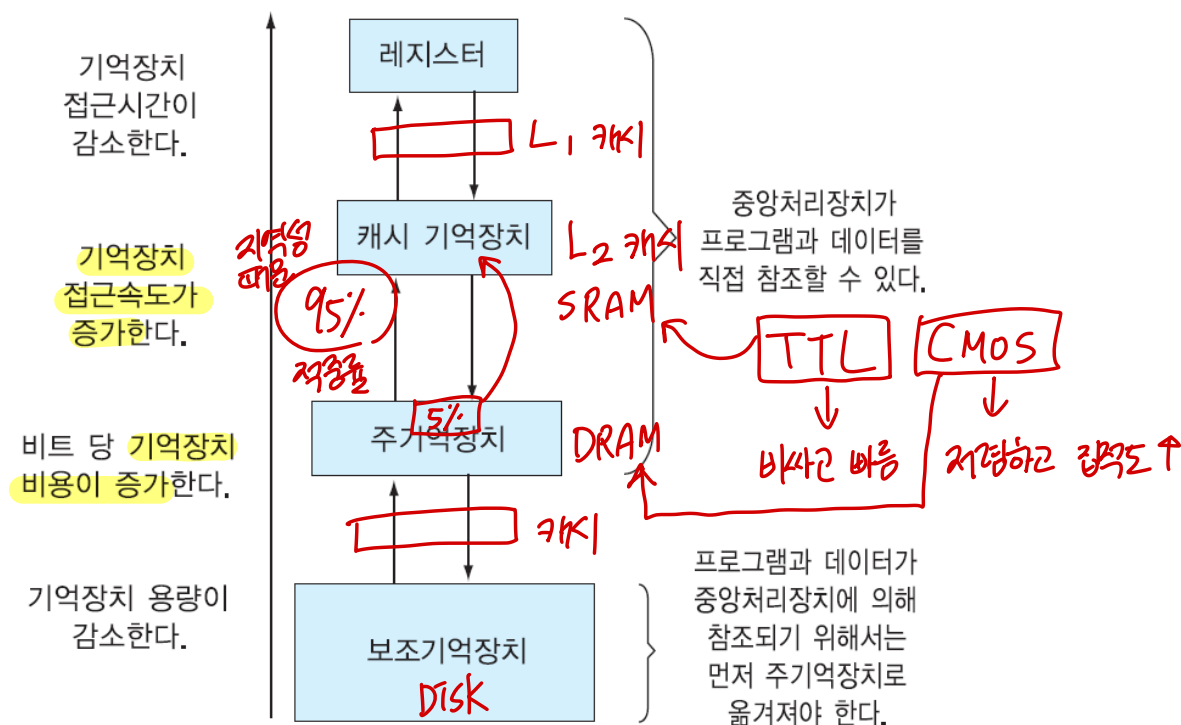


그림 3-3 기억장치의 계층 구조

기억장치의 계층 구조 및 관리 기법

- 기억장치 관리 기법

5주차 ch03 기억장치 관리

- ① 인출(fetch) 기법

- 주기억장치에 적재할 다음 프로그램이나 데이터를 언제 가져올 것인가를 결정함

page fault

- ① 요구 인출(demand fetch) 기법

- ~~2~~ 예상 인출(anticipatory fetch) 기법

- 배치(placement) 기법

- 새로 인출된 데이터나 프로그램을 주기억장치의 어디에 위치시킬 것인가를 결정하는 기법

- 1. 최초 적합(first-fit), 2. 최적 적합(best-fit), 3. 최악 적합(worst-fit)

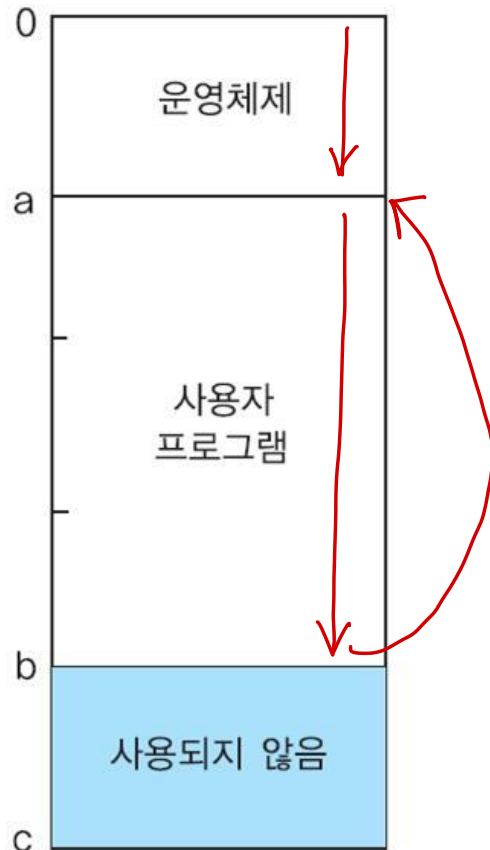
- 교체(replacement) 기법

- 새로 들어온 프로그램이 들어갈 장소를 마련하기 위해서 어떤 프로그램 및 어떤 데이터를 제거할 것인가를 결정

단일 사용자 연속 기억장치 할당

- 단일 사용자 연속 기억장치 할당
 - 초기의 컴퓨터 시스템에서는 임의의 시간에 한 사용자만이 시스템을 사용
 - 오버레이(overlay) 기법
 - 주기억장치보다 더 큰 프로그램의 실행이 가능
 - 시스템 보호
 - 중앙처리장치 내에 하나의 경계 레지스터(boundary register)를 이용
 - 슈퍼바이저 호출(SVC : Super Visor Call) 명령
 - 사용자가 운영체제에 서비스를 요청

단일 사용자 연속 기억장치 할당



경계 레지스터

사용자 프로그램이 계산한 모든 주소는 반드시 a보다 큰가를 확인하기 위해 검사된다.

segment fault

단편

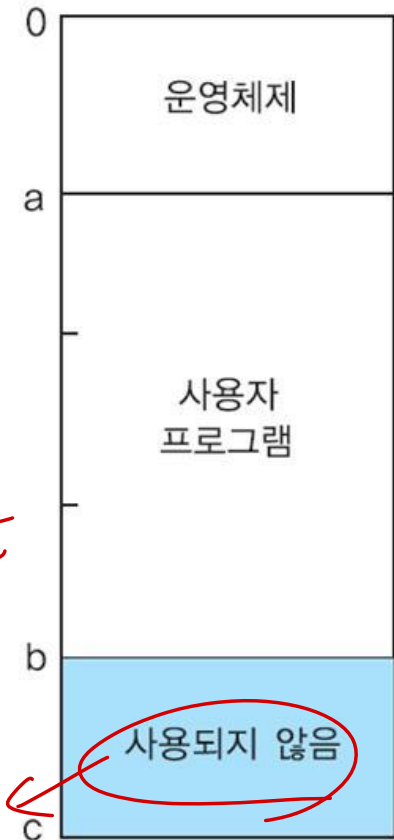


그림 3-4 단일 사용자 연속 기억장치 할당

그림 3-5 단일 사용자 연속 기억장치 할당에서 기억장치의 보호

3.4

고정 분할 기억장치 할당

- 주기억장치를 효율적으로 나누어 사용
- 다중 프로그래밍을 위하여
 - 여러 개의 작업이 동시에 컴퓨터의 주기억장치 내에 존재 해야 함
 - 주기억장치를 일정 수의 고정된 크기들로 분할하여 실행 중인 여러 프로세스에게 할당
- 1. 절대 번역 및 로딩
 - 지정된 분할에만 적재됨
- 2. 재배치 가능 번역 및 로딩
 - 어떤 프로그램이 수행될 시점에 수행에 충분한 크기의 분할을 선택하여 적재함
- 시스템 보호
 - 기억장치의 공유 시 한 분할에 있는 정보를 다른 분할에 있는 정보로부터 보호, 다수의 경계 레지스터(boundary register)를 이용

고정 분할 기억장치 할당

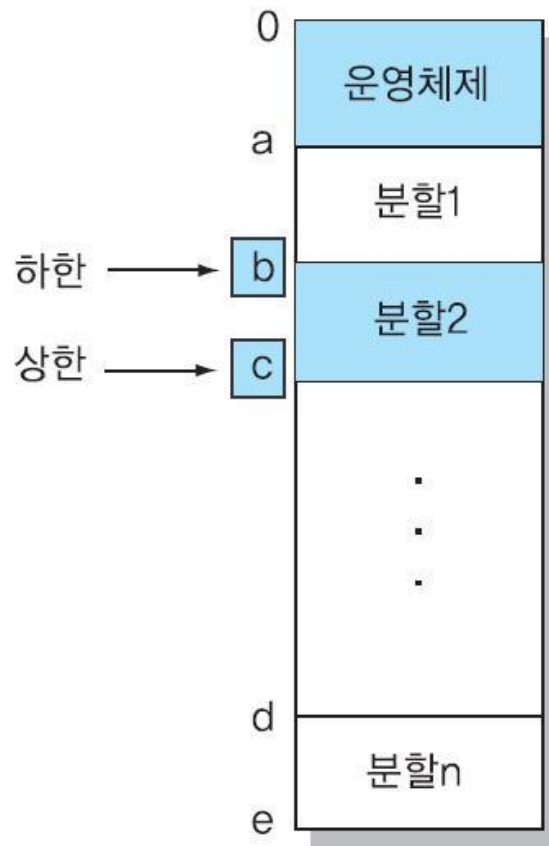


그림 3-6 다중 프로그래밍 시스템에서의 프로그램 및 데이터 보호

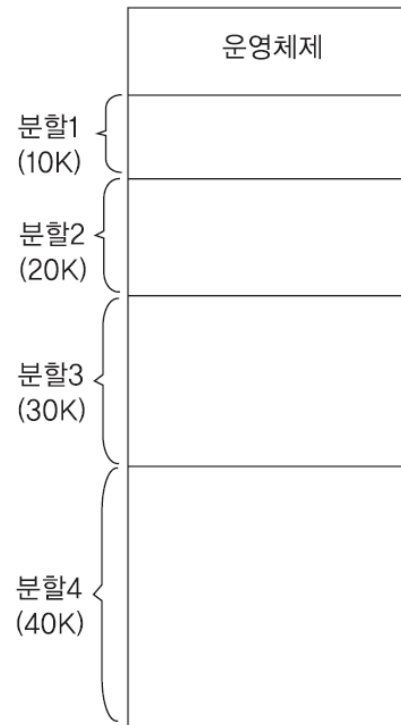
고정 분할 기억장치 할당

- 분할 공간의 크기와 단편화(fragmentation) 현상
 - 사용자 작업의 크기가 분할에 정확히 맞지 않거나
 - 분할이 너무 작아서 대기 중인 어떤 작업도 이 분할에 적재될 수 없는 경우에 발생
- 1. 내부 단편화(internal fragmentation)
 - 분할 내 미사용 부분 발생 시
- 2. 외부 단편화(external fragmentation)
 - 어떤 분할이 대기 중인 작업에게 너무 작아서 사용할 수 없을 때

고정 분할 기억장치 할당

작업1 : 37K
작업2 : 20K
작업3 : 25K

작업4 : 13K



(a) 초기 상태



(b) 작업1, 2, 3이 시작된 상태

그림 3-7 기억장치의 단편화 현상

3.5 가변 분할 기억장치 할당

- 작업들이 필요로 하는 만큼의 공간을 동적으로 할당
- 기억 공간의 효율화

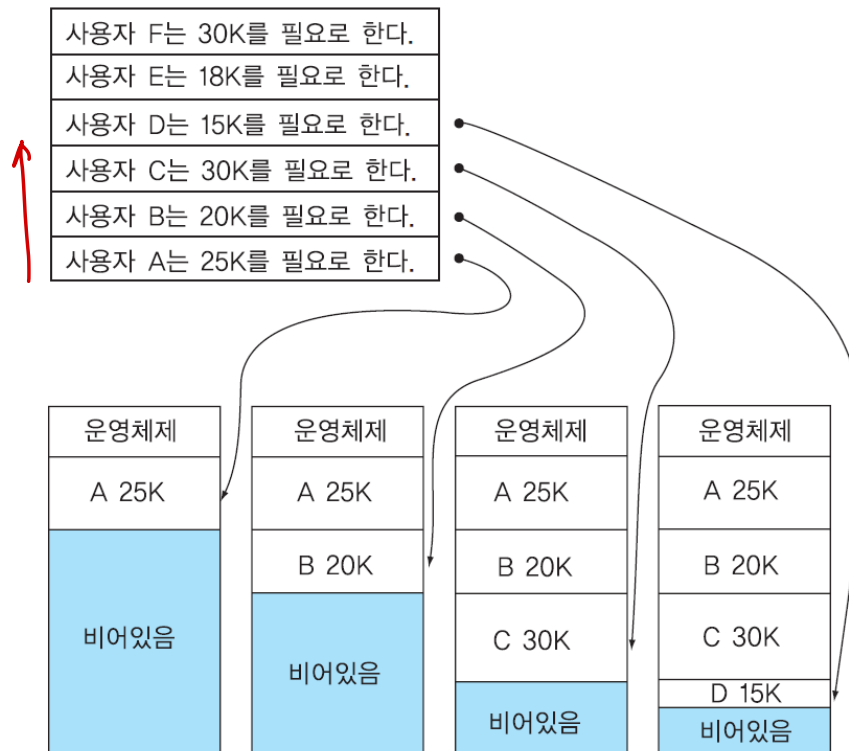


그림 3-8 가변 분할 다중 프로그래밍에서의 초기의 분할 할당

가변 분할 기억장치 할당

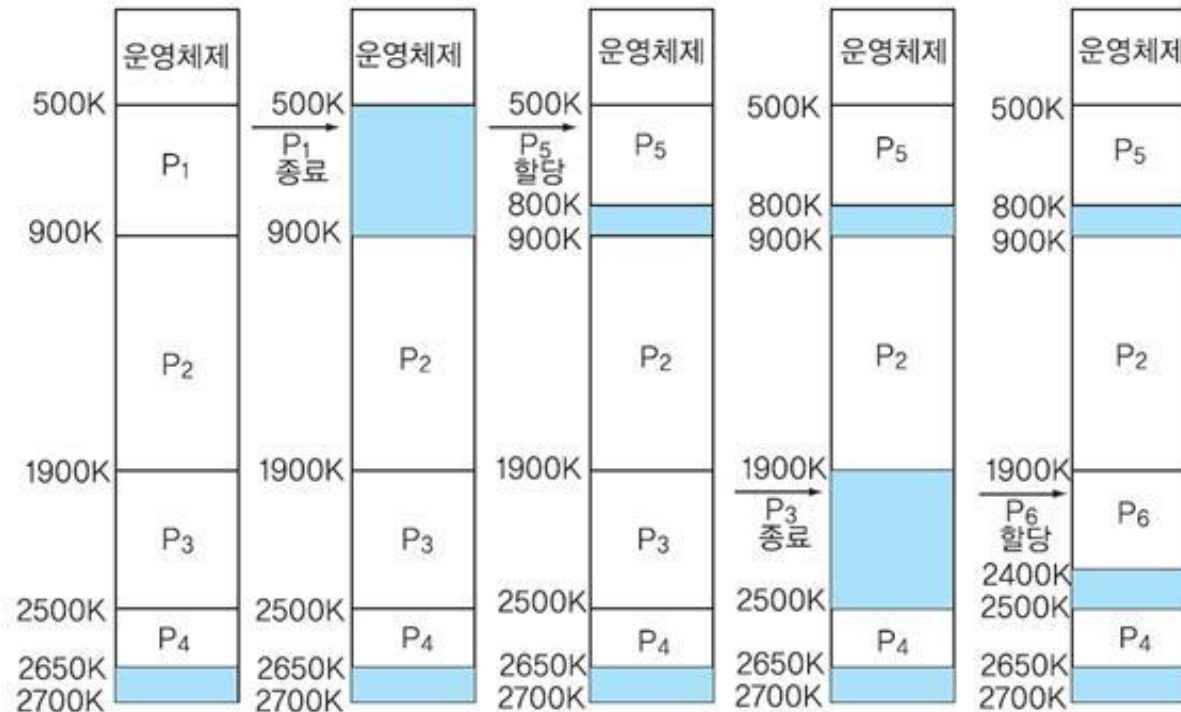


그림 3-9 가변 분할 다중 프로그래밍에서의 단편화 현상

가변 분할 기억장치 할당

- 공백의 합병(coalescing holes)
 - 인접한 공백들을 결합하여 하나의 큰 기억 공간으로 만드는 과정

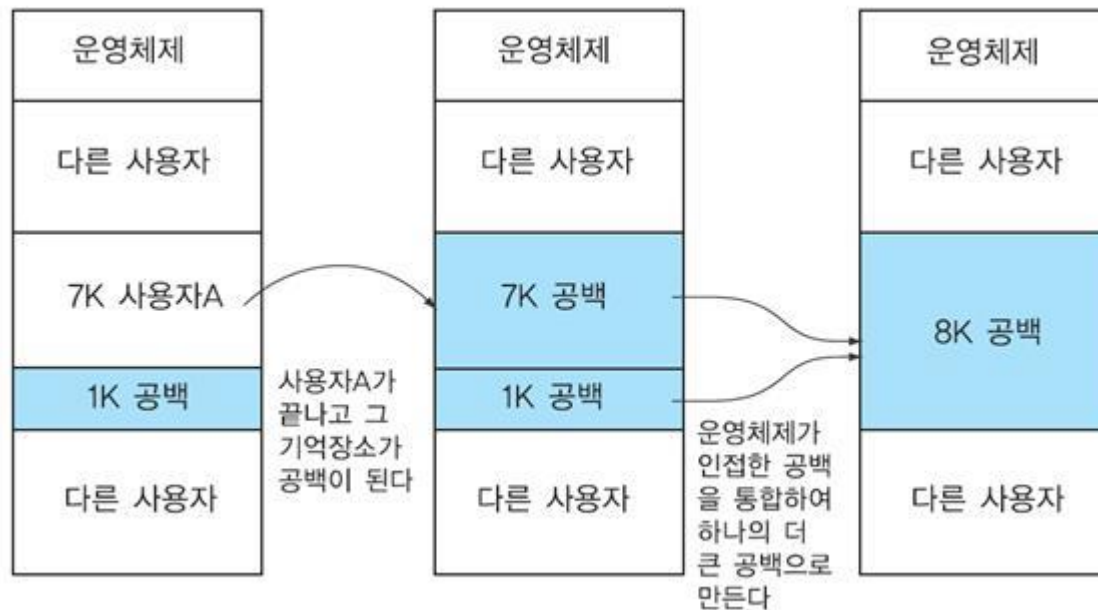


그림 3-10 가변 분할 다중 프로그래밍에서의 공백의 합병

가변 분할 기억장치 할당

- 기억장소의 집약(compaction)
 - 존재하는 여러 개의 작은 공백들을 하나의 커다란 기억 공간으로 통합
 - 일반적으로 쓰레기 수집(garbage collection)이라고 부르기도 함

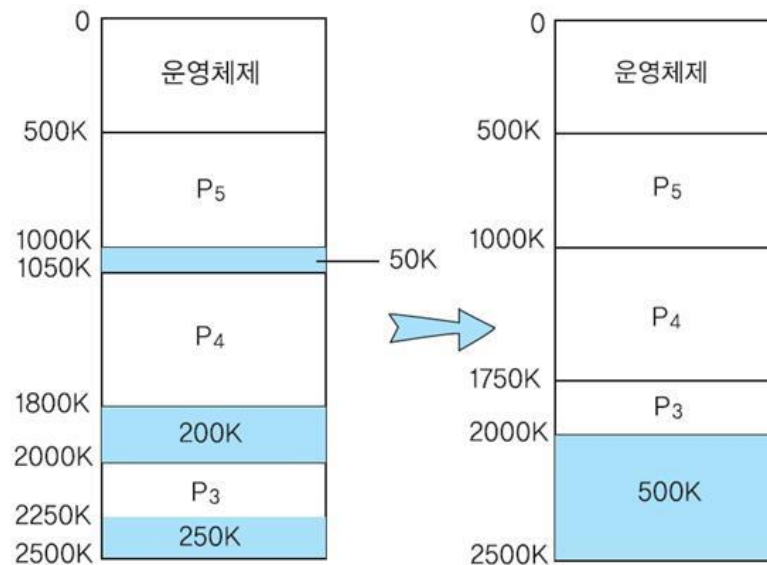


그림 3-11 기억장소의 집약

가변 분할 기억장치 할당

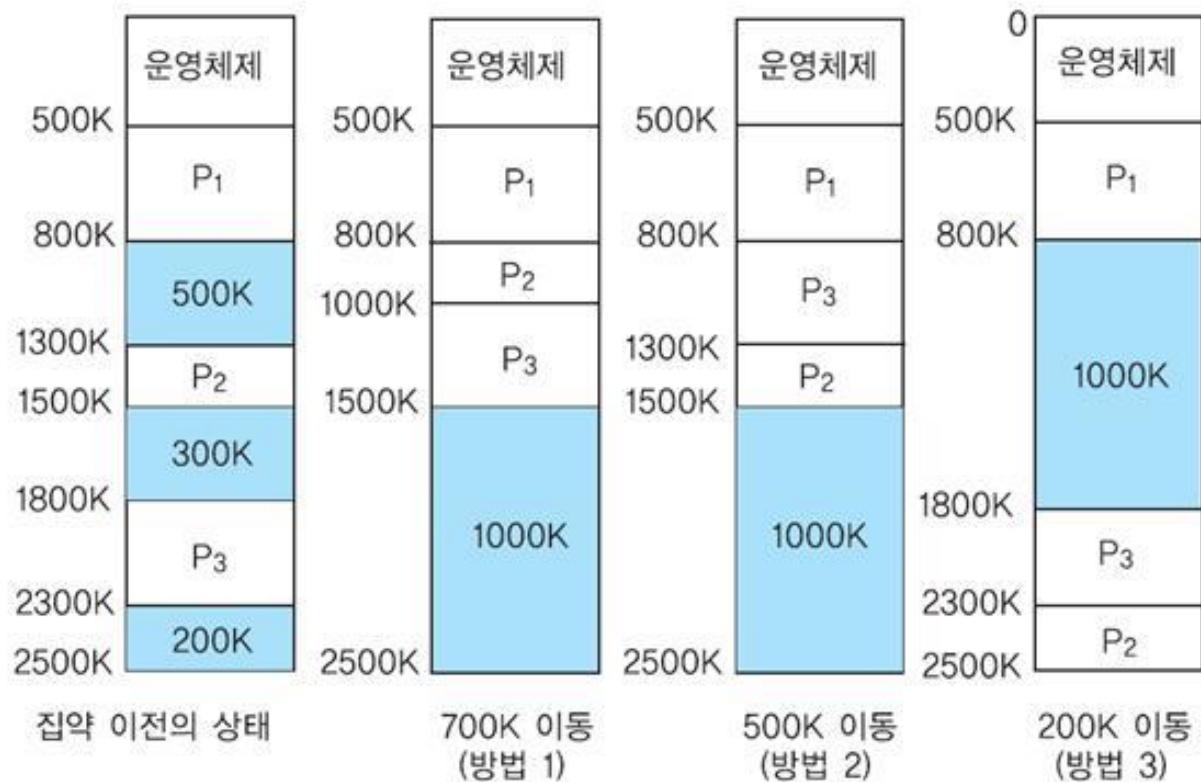


그림 3-12 기억장소의 집약 방법에 따른 비교

가변 분할 기억장치 할당

- 기억장치 배치 전략(memory placement strategy)
 - 최초 적합 기법(first-fit strategy)
 - 주기억장치의 첫 번째 유용한 공백을 우선적으로 선택
 - 널리 사용
 - 최적 적합 기법(best-fit strategy)
 - 가장 적합한 공간을 선택
 - 기억장치의 단편화를 최소로 하는 방법
 - 최악 적합 기법(worst-fit strategy)
 - 가장 큰 공백에 배치

가변 분할 기억장치 할당

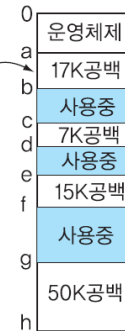
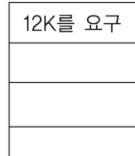
(a) 최초 적합(first-fit) 기법

빈 기억장치 리스트(free storage list)에서 그 작업을 수용할 수 있는 첫 번째 공백에 작업을 배치한다.

빈 기억장치 리스트

(기억장소 주소의 순서에 따르거나 때로는 임의의 순서로 배열)

시작주소	길이
a	17K
c	7K
e	15K
g	50K



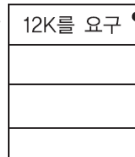
(b) 최적 적합(best-fit) 기법

넣을 수 있는 가장 작은 공백에 작업을 배치한다.

빈 기억장치 리스트

(공백의 크기 순서로 배열)

시작주소	길이
c	7K
a	15K
e	17K
g	30K



(c) 최악 적합(worst-fit) 기법

가장 큰 공백에 배치한다.

빈 기억장치 리스트

(공백크기의 역순으로 배열)

시작주소	길이
g	50K
a	17K
e	15K
c	7K

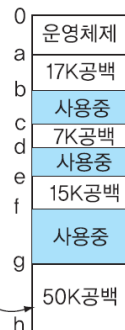
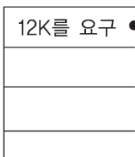


그림 3-13 최초 적합, 최적 적합, 최악 적합 기억장치 배치 기법

3.6 기억장치 교체

- 기억장치 교체(swapping)
 - 하나의 작업이 전체 기억장치를 사용한 후, 필요에 따라 그 작업은 제거(swap out)되고 다시 다음 작업을 적재(swap in)
 - 오늘날 일반적으로 사용되는 페이징 시스템(paging system)의 기초가 됨

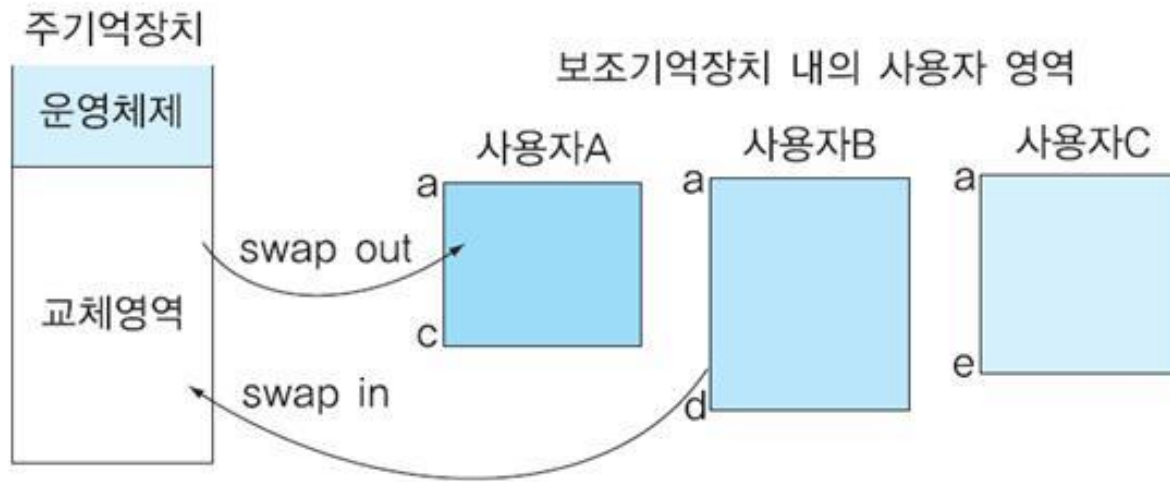


그림 3-14 다중 프로그래밍 시스템에서의 기억장치 교체